Date

Expt. No. 9. Page No. 8

Aim:- To implement a parallelized merge sort algorithm to sort a given set of elements, determine the time required for different values of n, & plot a graph of taken versus n.

Theory :- Merge sort is a divide & conquer sorting algorithm.

It divides the array into two ~~values~~ halves, recursively sort both halves & finally merge them.

Rule => Sort Left → Sort Right → Merge

Algorithm

```
merge (arr, low, mid, high)
    Create temprory array Temp
    i ← low
    j ← mid+1
    K ← 0

While i <= mid AND j <= high DO
        if A[i] <= A[j] then
            Temp[K] ← Arr[i]
            i ← i+1
        else
            Temp[K] ← Arr[j]
            j ← j+1
        end if
    K ← K+1
end while
```

Teacher's Signature

```
while j <= high do
    Temp [k] ← A[j]
        j ← j+1
        k ← k+1
    end while

for i ← 0 to k-1 do
    Arr[low+i] ← Temp[i]
    end for
end merge.
```

C++ Code

```
#include <iostream>
#include <vector>
using namespace std;
void merge(vector<int>& arr, int left, int mid,
                    int right) {
  int n1 = mid - left + 1;
  int n2 = right - mid;
  vector<int> L(n1);
  vector<int> R(n2);

for(int i=0; i<n1; i++) L[i] = arr[left+i];
for(int j=0; j<n2; j++) R[j] = arr[mid+1+j];
```

```
int i = 0;
int j = 0;
int k = left;

while (i < n1 && j < n2) {
    if (L[i] <= R[j]) {
        arr[k] = L[i];
        i++;
    } else {
        arr[k] = R[j];
        j++;
    }
    k++;
}
while (i < n1) {
    arr[k] = L[i];
    i++;
    k++;
}
while (j < n2) {
    arr[k] = R[j];
    j++;
    k++;
}
}
void mergeSort(vector<int>& arr, int left, int
        right) {
    if (left >= right) return;
```

```
int mid = left + (right - left)/2
mergeSort(arr, left, mid);
mergeSort(arr, mid+1, right);

merge(arr, left, mid, right);
}

void printVector(const vector<int>& arr){
    for(int num : arr){
        cout << num << " ";
    }
    cout << endl;
}

int main(){
    vector<int> data = {38, 27, 43, 3, 9, 82, 10};
    cout << "Original array";
    printVector(data);

    mergeSort(data, 0, data.size()-1);
    cout << "Sorted Array: ";
    printVector(data);
    return 0;
}
```

Time Complexity

Best Case : O(n log n)
Average Case : O(n log n)
Worst Case : O(n log n)

Space Complexity

O(n)

Teacher's Signature _____

Date

Expt. No. Page No. 12.

Result Thus, Parallel merge sort was successfully implemented using multiple threads. The execution time was measured for different input sizes & the relationship between the number of elements & execution time was delay analyzed using graph.

Viva Question

Q1 What is merge Sort?

Ans Merge sort is a divide & Conquer based sorting algorithm that divides the array into smaller parts, sorts then merges the sorted parts.

Q2 What is the time complexity of merge sort?

Ans O(n log n) for best, average & worst case.

Q3 Is merge sort stable?

Ans Yes, the standard implementation of Merge Sort is stable.

Q4 Why is parallel merge sort used?

Ans To execute independent sorting operations simultaneously & potentially reduce execution time on multi-core process.

Q5 What is a thread?

Ans A thread is the smallest unit of execution within a process.

Teacher's Signature